

RESEARCH PROJECT — CMI INFORMATIQUE

Creation of an Intelligent Assistant for First-Year Courses

Target audience: Everyone

Supervisor: Yannick Estève

A. Louvet L. Ouelhadj T. De Faveri-Serre C. Cloupet H. Baudey

Academics Years 2025–2027

This project aims to one one hand make some research contributions about LLMs, RAG, vector search and explain it regardless of prior familiarity with the concept.
and orchestration frameworks in the context of educational assistants, in the order build a conversational AI assistant grounded in Retrieval-Augmented Generation (RAG), capable of answering students' questions based on their own course documents.

Contents

1. State of the Art : Definitions and Context

1.1. Setting the Stage with Artificial Intelligence

1.1.1 A Brief History

The idea of machines that can think is not new. As early as 1950, Alan Turing proposed his famous *Turing Test* as a criterion for machine intelligence [?]: if a machine can sustain a conversation indistinguishable from a human, it could, in some sense, be said to think. The term “Artificial Intelligence” itself was coined in 1956 by John McCarthy at the Dartmouth Conference, which is widely regarded as the founding moment of the field.

The decades that followed alternated between periods of optimism and so-called “AI winters” — intervals of reduced funding and interest caused by the gap between ambitious promises and modest results. The landscape shifted dramatically in the 1980s and 1990s with the rise of *machine learning*: rather than hard-coding rules into a program, researchers began designing systems that could *learn* patterns directly from data. This paradigm shift laid the foundations for everything that followed.

The true turning point came in 2012, when a deep neural network called AlexNet [?] crushed the competition in the ImageNet visual recognition challenge, demonstrating that *deep learning* — neural networks with many layers trained on large datasets — could achieve performance far beyond classical methods. From that moment, deep learning became the dominant approach across almost every subfield of AI.

1.1.2 So What Is Artificial Intelligence?

The term *Artificial Intelligence* is one of the most used — and most misused — expressions in today’s technological landscape. Before discussing the specific systems at the heart of this project, it is worth taking a step back to understand what AI actually is, where it comes from, and why it has become so central to modern computing.

At its core, Artificial Intelligence refers to any computational system designed to perform tasks that would normally require human intelligence — such as recognizing objects in an image, understanding a spoken sentence, or deciding the next move in a chess game. A more precise definition, proposed by Russell and Norvig in their seminal textbook [?], describes an AI agent as a system that *perceives its environment through sensors and acts upon it through actuators in order to maximize a performance measure*.

A useful distinction is that between *narrow AI* (also called weak AI) and *general AI* (or strong AI). Narrow AI — which covers virtually all systems in existence today — is designed to excel at one specific type of task: a spam filter classifies emails, a recommendation engine suggests movies, a language model generates text. General AI, by contrast, would be a system capable of reasoning across arbitrary domains at a human level; it remains largely hypothetical and is an active subject of long-term research. Every AI system discussed in this project falls squarely in the narrow AI category.

1.2. A closer look at Large Language Models (LLMs)

ceceeedvfiej (à refaire)

1.2.1 Limitations: Hallucinations and Knowledge Boundaries

Despite their impressive performance, LLMs suffer from several well-documented limitations. The most critical one in an educational context is the tendency to produce *hallucinations* — generating text that is fluent and plausible but factually incorrect or unfounded [?, ?].

Hallucinations arise from multiple causes. First, LLMs rely on statistical correlations in their training data rather than grounded factual reasoning. Second, their knowledge is static: they have a training cutoff date and cannot access new or domain-specific information unless explicitly provided. Third, when a query falls outside the model’s confident knowledge region, it may confabulate an answer rather than acknowledging uncertainty. According to a 2024 survey published in the *ACM Transactions on Information Systems*, hallucinations are categorized into two main types: *factuality hallucination* (stating incorrect facts) and *faithfulness hallucination* (producing content inconsistent with a given source).

These limitations are particularly problematic in an educational context, where accuracy is essential. An assistant that confidently provides wrong information about a course concept could actively mislead students. This motivates the use of external document grounding through RAG.

1.2.2 Biase Considerations Issue : Feedback Loop Degradation via RLHF

A subtler but significant limitation concerns the feedback mechanism used to improve LLMs by using techniques such as *Reinforcement Learning from Human Feedback* (RLHF) and Constitutional AI are used to align model behavior with human preferences and reduce harmful outputs. This system bases on a system in which users rate model responses positively or negatively and the model is updated accordingly [?]. While this process is intended to align model behavior with user expectations, it can produce perverse effects when users consistently submit poorly formulated prompts.

When a user provides an ambiguous, incomplete, or incoherent query, the model has no reliable basis on which to produce a relevant answer. Then it generates a response that is statistically plausible given the input, but which inevitably fails to match the user’s unstated intent. The user, perceiving the response as irrelevant or absurd, assigns a negative rating — even though, from the model’s perspective, the response was a reasonable completion of the given prompt. This misalignment between the signal (a negative rating) and its actual cause (a poorly formulated query rather than a model error) introduces *corrupted supervision* into the training loop.

Repeated at scale, this dynamic causes the model to associate certain legitimate response patterns with negative reward and to unlearn them, leading to a measurable degradation in general response quality over time. HAS study by Chen et al. [?] empirically documented this phenomenon for ChatGPT, observing significant behavioral shifts between model versions, with certain tasks where earlier versions outperformed later ones. The authors attribute part of this regression to the compounding effects of user feedback noise in the RLHF pipeline.

This issue is particularly relevant in an educational setting, where students — especially at the first-year level — may lack the prompt-engineering literacy to formulate precise queries. An assistant that degrades under noisy feedback could thus become progressively less useful for its primary audience, underscoring the importance of designing robust feedback collection mechanisms and, where possible, reducing reliance on unfiltered user ratings for model updates.

1.3. Let’s recap

A natural starting point for this overview is a question that seems simple but is actually quite nuanced: what exactly is an LLM, and how does it differ from the broader notion of “AI”? The term *Artificial Intelligence* (AI) is an umbrella concept encompassing any technique or system that enables machines to mimic human intelligence, including machine learning, computer vision, robotics, and natural language processing. A *Large Language Model* (LLM), by contrast, refers to a specific category of AI model designed to understand and generate human language. LLMs are a subset of AI, just like *Machine Learning* (ML) or *Deep Learning* (DL) etc... while all LLMs are AI systems, the converse is not true.

Concretely, LLMs are neural networks trained on massive text corpora using self-supervised learning, based on the Transformer architecture introduced by Vaswani et al. [?], which enables models to capture long-range dependencies in text through self-attention mechanisms. The subsequent introduction of OpenAI’s GPT series — GPT-2 in 2019, GPT-3 in 2020, and later GPT-4 — demonstrated that scaling up model size and training data leads to *emergent capabilities* in language understanding and generation [?]: abilities that appear qualitatively only beyond a certain scale threshold. Contemporary LLMs such as GPT-4, Anthropic’s Claude, Meta’s LLaMA, Google’s Gemini, and Mistral exhibit remarkable proficiency across a wide range of tasks — from question answering and summarization to code generation and multi-step reasoning — and are characterized by parameter counts in the tens or hundreds of billions. These models operate by encoding vast amounts of world knowledge into their parameters through statistical pattern learning over billions of tokens, and their trajectory of development has been defined by increasing model size, diversifying training data, and refining training techniques.

1.4. Retrieval-Augmented Generation (RAG)

1.4.1 Principle and Motivation

Before diving into the technical architecture, it is worth stepping back to explain what RAG is and why it was developed, so as to make this research accessible and interesting to everyone.

Imagine asking a very knowledgeable friend a question about a specific topic — say, the content of a course they have never attended. No matter how smart they are, they simply cannot know what was taught in that particular class. An LLM faces the exact same limitation: it only knows what it saw during training, and it has no way to consult documents it has never encountered. Now imagine instead that before answering, your friend is allowed to quickly flip through the relevant course notes and use what they find to inform their reply. That, in essence, is what RAG does: it gives the model access to a specific set of documents at the moment of answering, so that its response is grounded in those sources rather than in its general (and potentially outdated or incomplete) memory.

More formally, Retrieval-Augmented Generation (RAG) is a paradigm that enhances LLMs by connecting them to an external knowledge base at inference time [?]. Rather than relying solely on the model’s parametric memory, a RAG system first retrieves relevant document passages from a curated corpus based on the user’s query, then passes these passages as context to the LLM, which generates a grounded response.

This approach directly addresses the limitations described in the previous section: it provides the LLM with up-to-date and domain-specific information, and it anchors the generation in explicit, verifiable sources — significantly reducing the risk of hallucination. A survey specifically focused on RAG for educational applications [?] confirms that this approach substantially improves the relevance and factual accuracy of AI-generated answers in academic settings.

1.4.2 RAG Architecture

A standard RAG pipeline consists of three stages:

1. **Indexing (developer phase):** Source documents (PDFs, course slides, lecture notes, etc.) are split into chunks, encoded as vector embeddings using an embedding model, and stored in a vector database. (*cf Section 1.4 with details about vector search and indexing*).
2. **Retrieval (user phase):** When a user submits a query, it is encoded into the same embedding space. The system then performs a similarity search to retrieve the top- k most relevant chunks from the database.
3. **Generation:** The retrieved chunks are prepended to the user query as context in the LLM's prompt. The LLM then generates a response grounded in this retrieved information.

1.4.3 RAG Variants and Evolution

The field has matured significantly since the original RAG paper. Surveys from 2024–2025 identify three main paradigms [?]:

- **Naive RAG:** The basic retrieve-then-generate pipeline described above.
- **Advanced RAG:** Introduces query rewriting, re-ranking of retrieved results, and iterative retrieval to improve precision.
- **Modular RAG:** A more flexible architecture where each component (retriever, reader, memory, etc.) can be independently swapped or optimized.

More recent work also explores **Agentic RAG**, where the model can decide *when* to retrieve and *which* sources to consult based on the complexity of the query, using a multi-step reasoning loop [?].

For our project, a Naive or Advanced RAG architecture is sufficient given the well-defined and bounded scope of the document corpus (first-year course materials).

1.5. Vector Search and Indexing

1.5.1 What Is an Embedding?

An *embedding*, in the context of artificial intelligence and large language models (LLMs), refers to the transformation of a piece of information — a word, a sentence, a document, or sometimes even an image — into a sequence of real numbers called a *vector*. This numerical representation allows the machine to manipulate language and concepts no longer as symbolic text, but as mathematical objects on which it can compute.

Rather than “understanding” words the way a human does, the model places them in a high-dimensional *vector space* (typically between 384 and 4096 dimensions depending on the model), where each item occupies a precise position. In this space, the *distance* between two vectors reflects their *semantic similarity*: two sentences with similar meanings will have nearby vectors, even if they share no words in common. This is the principle that allows a system, for instance, to retrieve the passage “*photosynthesis allows plants to produce energy*” from the query “*how do plants generate their energy?*”.

1.5.2 How Are Embeddings Learned?

These vectors are not chosen arbitrarily: they are *learned* during the training of the model. The system reads enormous quantities of text and progressively adjusts its parameters so that the resulting vectors capture regularities of language. For example, the model learns that “cat” and “dog” appear in similar contexts and therefore ends up assigning them nearby vectors, while “car” will sit further away.

Early work such as *Word2Vec* [?] and *GloVe* [?] showed that this simple principle is enough to give rise to remarkable semantic relationships. The canonical example of vector arithmetic, $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$, illustrates that the learned space encodes concepts (gender, royalty) rather than mere co-occurrences.

In modern Transformer-based models, embeddings are also *contextual*: the vector associated with a word depends on the sentence in which it appears. The word “bank” in “*I sat by the river bank*” and in “*I deposited money at the bank*” thus receives two distinct representations, which naturally resolves lexical ambiguities. For semantic search, one typically relies on specialized models such as those from the **sentence-transformers** family [?], which produce a *single vector* per sentence or paragraph, optimized for similarity comparison rather than for text generation.

1.5.3 Measuring Similarity Between Vectors

Once content is represented as vectors, comparing two pieces of text amounts to comparing two points in space. Three metrics are commonly used:

- **Cosine similarity**, which measures the angle between two vectors and is insensitive to their magnitude: by far the most widely used metric in semantic search.
- **Dot product**, faster to compute but sensitive to vector norms; preferred when vectors have been normalized beforehand.
- **Euclidean distance**, intuitive but often less discriminating in very high dimension because of the so-called *curse of dimensionality*.

1.5.4 Indexing and Approximate Nearest Neighbor Search

When a corpus contains thousands or even millions of indexed passages, comparing the query against every stored vector — the so-called *exact*, or “brute force”, search — becomes prohibitively expensive. The standard solution is to rely on *indexing* structures that enable *Approximate Nearest Neighbor* (ANN) search: one trades a small amount of accuracy for several orders of magnitude in speed.

Several indexing algorithms have become standard:

- **IVF (Inverted File Index)**: partitions the space into cells via a preliminary clustering step (typically *k*-means), then restricts the search to the cells closest to the query.
- **HNSW (Hierarchical Navigable Small World)** [?]: builds a multi-layer graph in which local navigation rapidly converges towards the neighborhood of the queried vector. It is currently the reference algorithm in most vector databases, thanks to its excellent trade-off between recall and latency.
- **Product Quantization (PQ)**: compresses vectors by sub-quantizing each subspace, drastically reducing memory usage at the cost of a slight accuracy loss.

These algorithms are implemented in dedicated libraries and databases — the so-called *vector databases* — the most widely used in the open-source ecosystem being **FAISS** (Facebook AI Similarity Search), **Chroma**, **Qdrant**, and **Weaviate**. The choice depends mainly on the volume

of data, latency constraints, and whether one needs metadata filtering (for example, restricting the search to a specific course or chapter).

1.5.5 Summary

In practice, embeddings are the foundational building block of any RAG system: they are used to interpret queries, to compare course passages, to perform semantic search, and more broadly to feed the internal mechanisms of LLMs. They can be seen as a “translation” of human language into a geometry of concepts, where ideas are organized in a multidimensional space that the machine can efficiently explore and exploit. It is precisely this geometry, combined with a suitable index, that will allow our educational assistant to retrieve in a few milliseconds the course passages relevant to answering a student’s question.

2. Project Overview

2.1. Pedagogical Objectives

The goal is to discover the fundamental principles of modern artificial intelligence through large language models (LLM) and the RAG (Retrieval-Augmented Generation) system. Students will learn how to interface an LLM with a custom document base to create a conversational assistant capable of answering questions about their courses.

2.2. AI Assistants in Educational Settings

2.2.1 Overview of Existing Work

The application of AI-powered assistants to education has attracted growing research interest. A survey published in *Applied Sciences* [?] identified 47 studies on RAG-based educational chatbots, the majority published in 2024, demonstrating the rapid growth of this field.

Several real-world deployments are directly relevant to our project:

- **UniMiB Assistant** (Bicocca University, Milan, 2024) [?]: A RAG-based student assistant capable of answering questions about university procedures, course schedules, and administrative information. It indexes official university documents and uses an LLM to generate answers grounded in those sources.
- **OwlMentor** (Frontiers in Psychology, 2024) [?]: An AI-powered learning environment designed to help university students comprehend scientific texts. Integrated into a course, it offered document-based chat, automatic question generation, and quiz creation.
- **CS50 AI Assistant** (Harvard University): Integrated into an introductory computer science course, a RAG-based assistant was shown to provide detailed and contextually appropriate responses, facilitating personalized tutoring at scale.
- **Medical education RAG assistant** (Nature *npj Digital Medicine*, 2025) [?]: A teaching assistant deployed in a medical school course, where usage patterns and student feedback were analyzed across two consecutive cohorts, demonstrating the feasibility and effectiveness of RAG-based assistants in high-stakes educational contexts.

2.2.2 Effectiveness and Challenges

The general finding across these studies is that RAG-based assistants are effective at providing accurate, context-grounded answers to domain-specific questions, and that students perceive them positively as a complement to human instruction. However, several challenges are consistently reported:

- **Document quality and coverage:** The quality of the assistant's answers is directly bounded by the quality and coverage of the indexed document corpus. Poorly formatted or incomplete course documents degrade performance.
- **Out-of-scope queries:** Students often ask questions outside the domain of the indexed documents. Robust systems must detect such queries and gracefully decline to speculate.
- **Evaluation difficulty:** Assessing the correctness of generated answers in open-ended educational contexts is non-trivial and often requires human expert evaluation.
- **Student trust calibration:** Students may over-rely on AI answers or, conversely, distrust them even when correct. Communicating uncertainty and citing sources is important for appropriate trust calibration.

2.3. Summary and Positioning of Our Project

This state of the art reveals that the combination of LLMs and RAG represents the current best practice for building domain-specific conversational assistants. The technology stack we plan to use — Python, an embedding model (`sentence-transformers` or OpenAI), a vector database (FAISS or Chroma), and an orchestration framework (LangChain or LlamaIndex) — is well-validated both in the academic literature and in real-world educational deployments.

The main originality of our project lies in its specific scope: building an assistant tailored to *first-year CMI Informatique course materials*, which have specific characteristics (introductory-level computer science, mathematics, and project content in French and English). The challenges we anticipate align with those reported in the literature: ensuring sufficient document coverage, managing out-of-scope questions, and calibrating student trust in the system’s outputs.